

A slow-verifier separation for Löb’s rule

An affirmative construction for the chosen-system clause of MAIS-O11(1)

Research note for Andrew Hagy
AI-assisted mathematical draft; no Lean certificate

22 September 2026

Abstract

Assuming the consistency of Peano arithmetic, we construct a consistent, recursively presented extension S of PA, with binary numerals and nested, priced abbreviations, and sentences p_n such that

$$\ell_S(\Box_S p_n \rightarrow p_n) = O(\log(n + 2)), \quad \ell_S(p_n) > n/C - 1.$$

Consequently, after taking a tail, this family satisfies the fixed-factor inequality in MAIS-O11(1), including its additive toll, with $\delta = 1$. The main construction uses a countable, effectively coded set of new nullary predicates and a primitive recursive verifier that performs very long finite searches. An arithmetic-only variant is also given, with its failure of uniform instantiation explicitly proved. It addresses the stated permission to choose an efficient system, where “efficient” has the three-clause meaning in MAIS-A1. It does not establish a speedup in the specified PA_{bin} , decide MAIS-O11(2), or give a polynomial-time implementation of the specified verifier.

Scope and verification status. This is a mathematical construction, with its reductions and assumptions spelled out below. Neither a byte-level implementation of its arithmetization nor a Lean formalization has been produced. No claim of historical novelty or independent expert verification is made. The deliberately slow checker is an essential part of the example, rather than a performance detail that can be discarded. The source question and its fixed-system polynomial-overhead conjecture must be kept distinct.

1 The precise target and the allowed freedom

Write $\ell_T(A)$ for the least number of written characters in a T -proof of A . All abbreviation definitions and names count. The requested lower bound is a family P_i with reflection lengths tending to infinity and

$$\ell_S(P_i) \geq 2\ell_S(\Box_S P_i \rightarrow P_i) + C_1(|P_i| + 1), \tag{1}$$

where C_1 is a fixed constant for the cheap transformation from a proof of P to a proof of $\Box_S P \rightarrow P$. This gives $\delta = 1$ in the problem.

MAIS-A1 calls a system efficient if it is a consistent recursively axiomatized extension of PA, has logarithmic-length numerals, and allows nested proof abbreviations. Its definition of a proof requires a primitive recursive checker. These requirements do not impose a polynomial running-time bound on the checker, nor do they require the language of an extension to have the same finite signature as PA. Both freedoms are used here.

The proof files below use an ordinary Hilbert calculus. There is no Löb inference rule built into the calculus and no extra padding charged to conclusions. The additional computational work takes place when the verifier checks whether the file is admissible.

2 A fixed base calculus and its hard finite sentences

Fix a Hilbert presentation B of PA with efficient binary numerals and nested abbreviations for complete terms and formulas. Choose a fixed finite basis of propositional axiom schemes, the usual quantifier and equality schemes, and PA induction, so checking an expanded proof takes polynomial time in its expanded length. One may implement binary constructors $b_0(t) = 2t$ and $b_1(t) = 2t + 1$ as a fixed definitional extension of arithmetic. Assume B is consistent.

All tokens, including variable and abbreviation identifiers, are represented over a fixed finite character alphabet. Identifiers use binary indices. An abbreviation stands for a term or formula, not a fragment of a token; in particular it cannot assemble a new predicate name from pieces of names. Definitions can refer to previous definitions. A verifier may expand the definitions and check the resulting Hilbert proof. This is primitive recursive, and PA proves termination and the usual elementary syntactic invariants of this specified procedure. Abbreviations need not be expanded when counting the length of the input file.

Let $\text{Prf}_B(u, v)$ be a natural, PA-provably Δ_1 representation of this checker, and let $\text{Len}(u)$ count written characters. By the parameterized diagonal lemma there is a fixed arithmetic formula $D(x)$ such that, writing $D_n = D(\bar{n})$,

$$B \vdash \forall x (D(x) \leftrightarrow H(x)), \quad (2)$$

where

$$H(n) :\iff \text{no } B\text{-proof of } D_n \text{ has written length at most } n. \quad (3)$$

Inside arithmetic the conclusion code in (3) is produced by the primitive recursive operation that substitutes the canonical numeral for n into the fixed formula D . Thus (2) is a uniform arithmetical theorem, not an informal substitution of a free variable into a quotation. The predicate H is decided by checking all proof files of length at most n ; it has a PA-provably Δ_1 representation. Also $|D_n| = O(\log(n + 2))$.

Lemma 1. *For every standard n , $H(n)$ is true, $B \vdash D_n$, and*

$$\ell_B(D_n) > n.$$

Proof. Suppose u were a B -proof of D_n of length at most n . Its existence would give $B \vdash D_n$. Numeralwise verification of the particular finite proof u would also give $B \vdash \neg H(\bar{n})$. But (2) and the proof of D_n give $B \vdash H(\bar{n})$. This contradicts consistency. Hence no such u exists and $H(n)$ is true. The exhaustive finite check establishing $H(n)$ has a terminating computation; PA can verify that computation, however long it is. Thus $B \vdash H(\bar{n})$, and (2) yields $B \vdash D_n$. $\square$

This is the familiar finite Gödel-sentence lower bound; see Pudlák's discussion of the diagonal sentences in [3, Section 3.1]. We have included its proof because it works for the chosen abbreviation calculus without importing a quantitative finite-consistency theorem from a different calculus. No bound on the time taken by the finite check is used to infer that its proof is short.

3 Constructing a verifier that refers to its own proof predicate

Expand the language by distinct nullary predicates

$$p_0, p_1, p_2, \dots$$

These are separate predicate symbols, not instances of a unary predicate $p(x)$. Encode the name p_n with a binary index, so $|p_n| = O(\log(n + 2))$. The characters in the name are charged, and names

cannot be formed by abbreviation expansion. Fix an injective primitive recursive formula coding with

$$\#p_n = 2n + 1. \quad (4)$$

Assign even codes to all other formulas using an ordinary string coding. The numeral for a formula's code still has size $O(|A| + 1)$. In particular, the canonical numeral for $\#p_n$ is $b_1(\bar{n})$. Proof-file codes can use the usual base-alphabet string encoding.

3.1 The parameterized checker

For a code e of an arithmetic formula $\vartheta(u, v)$ with two designated free variables, define $\text{Check}(e, \pi, y)$ by the following terminating algorithm:

1. Parse π as a Hilbert proof with the same abbreviation discipline as B , and require its expanded conclusion to have code y .
2. Allow the ordinary logical and PA axioms in the expanded language, modus ponens, generalization, and abbreviation definitions. In addition, allow the axiom instances

$$R_{e,n} := (\exists u \vartheta(u, \overline{2n+1})) \rightarrow p_n. \quad (5)$$

The axiom formula must occur in the proof file, possibly through previously defined abbreviations. Recognizing it only inspects the text coded by e ; it does not evaluate that formula.

3. For every primitive predicate p_j occurring in the file, including in abbreviation definitions, run the finite test $H(j)$ and require that it returns true. An invalid formula code e or malformed file is rejected.

The algorithm is primitive recursive. The last step performs bounded proof search in the already fixed base system B . There is no search for an unbounded proof and no recursive invocation of the new verifier.

In B one proves the syntactic invariant

$$B \vdash \forall e \forall \pi \forall n (\text{Check}(e, \pi, 2n+1) \rightarrow H(n)). \quad (6)$$

Indeed the only formula with code $2n+1$ is p_n . A file whose expanded conclusion is p_n must contain that literal predicate token in one of its records or definitions: expansion of complete terms and formulas does not manufacture new predicate tokens. Induction over the ordered abbreviation definitions proves this fact in PA. The verifier has therefore checked $H(n)$. This establishes (6) uniformly, without assuming that every $H(n)$ is true.

3.2 A fixed point for the checker description

Apply the parameterized diagonal lemma to the arithmetic relation $\text{Check}(e, \pi, y)$. Obtain an arithmetic formula $V(\pi, y)$, with code $v = \#V$, for which

$$B \vdash \forall \pi \forall y (V(\pi, y) \leftrightarrow \text{Check}(\bar{v}, \pi, y)). \quad (7)$$

It can be chosen PA-provably Δ_1 : the diagonal construction uses primitive recursive, PA-provably total substitution operations, and Check is such a relation. Equivalently, substitute the graph of the unique output of the diagonal substitution operation into its Δ_1 definition.

Now define an S -proof to be a file accepted by the concrete checker $\text{Check}(v, -, -)$. Define its arithmetized proof predicate to be V and put

$$\Box_S A := \exists \pi V(\pi, \ulcorner A \urcorner).$$

Equation (7) identifies this predicate with the actual checker inside PA. It is not merely an externally equivalent alternative provability predicate. The extra axioms (5) are now exactly

$$R_n = \Box_{Sp_n} \rightarrow p_n. \quad (8)$$

This removes the apparent circularity in saying that the extra axioms mention provability in the very system being defined.

3.3 A finite runtime bound for the verifier

The verifier is deliberately slow, but not an unspecified recursive search. There is a fixed integer $c \geq 1$ for which an implementation on proof files of length m computes the conclusion and acceptance result in at most

$$T(m) = 2^{2^{c(m+1)^2}}$$

steps. Comparing the conclusion against a separately supplied code also charges the length of that code; alternatively reject an oversized supplied code after reading a prefix longer than the maximal possible conclusion.

Here is a conservative bound sufficient for the claim. A file of length r has at most r ordered definitions. Each expansion stage increases the current length by a factor at most $r + 1$, so the expanded text has length at most $(r + 1)^{r+1} \leq 2^{(r+1)^2}$ for $r \geq 1$. There are at most $|\Sigma_B|^{j+1}$ candidate proof files of length at most j . Their expansion and polynomial-time checking therefore decide $H(j)$ within $2^{c_2(j+1)^2}$ steps for a fixed c_2 . In an m -character S -file, every tested index j has $O(m + 1)$ binary digits, and there are at most m distinct literal predicate names. Substitution into this bound, together with the ordinary syntax checks, is absorbed by $T(m)$ after increasing c . Malformed inputs can be rejected before the corresponding work is done.

All these bounds concern fixed finite loops, expansion, and arithmetic on finite strings; PA proves termination with an elementary bound by the same inductions. This is a runtime bound for the chosen implementation, not a polynomial-time claim or an explicit length certificate for internalizing the computation as an arithmetic proof.

Combining (6), (7), and (2) gives a *single fixed B-proof* of

$$\forall n \left((\exists \pi V(\pi, b_1(n))) \rightarrow D(n) \right). \quad (9)$$

This uniform theorem is the main reason the lower-bound interpretation below can be efficient even though the verifier is not.

4 Consistency, arithmetic strength, and Löb's theorem

Proposition 2. *The system S is a consistent, recursively presented extension of B . It is conservative over B for arithmetic sentences, has logarithmic numerals and nested priced abbreviations, and satisfies the Hilbert–Bernays–Löb derivability conditions for $\Box_S$. For each n , both R_n and p_n are S -theorems.*

Proof. Every B -proof is an S -proof: it contains no new predicate symbols, so the extra finite tests are vacuous. The new syntax and checker are effective and primitive recursive. By Lemma 1, all the tests $H(j)$ are true for standard j . Thus each one-line extra-axiom proof of R_n is accepted.

For consistency, interpret every p_j as the arithmetic tautology $0 = 0$, leaving all arithmetic unchanged. Every extra axiom becomes $\Box_{Sp_j} \rightarrow (0 = 0)$, a B -theorem. In this expression $\Box_{Sp_j}$ is an arithmetic formula containing the numeral $2j + 1$; its numeral is left unchanged by the interpretation. Logical axioms, PA induction instances, and inference rules translate to valid

B -derivations. Consequently an S -proof of an arithmetic sentence gives a B -proof of that same sentence. In particular an S -contradiction would contradict the consistency of B .

For completeness, the required derivability conditions hold for the specified verifier, rather than being assumed for an arbitrary predicate.

1. If $S \vdash A$, the acceptance computation of its particular finite proof can be verified in B . Hence $B \vdash \Box_S A$.
2. PA proves correctness of the primitive recursive proof-composition operation for modus ponens. Combining two accepted proof files introduces no new primitive predicate tokens; all the requisite tests $H(j)$ were already checked in the input files. This proves

$$S \vdash \Box_S(A \rightarrow B') \rightarrow (\Box_S A \rightarrow \Box_S B').$$

3. The formula $\Box_S A$ is arithmetical Σ_1 , using a Σ_1 presentation of V . Formalized Σ_1 -completeness in PA gives $\Box_S A \rightarrow \Box_B \Box_S A$. PA also proves that purely arithmetic B -proofs embed into S , since the added tests are vacuous for those files. It follows that $S \vdash \Box_S A \rightarrow \Box_S \Box_S A$.

The diagonal lemma is available in the expanded language, as usual for an extension of arithmetic. Löb's theorem therefore applies to S . Applying it to its theorem R_n proves $S \vdash p_n$. $\square$

The tests do not add certificates or padding to the written proofs. Combining and specializing proofs uses the underlying Hilbert syntax. In particular, appending $P \rightarrow (\Box_S P \rightarrow P)$ and a modus-ponens line gives the cheap-reflection inequality from MAIS-A1, with some fixed C_1 . The added formulas introduce no new p_j outside those already in P : the quotation in $\Box_S P$ is arithmetic text. One can omit explicit line references in an MP record and let the checker find the premises, so this argument does not assume that arbitrarily large line numbers have constant character cost.

5 The polynomial interpretation and the lower bound

Define the interpretation I by

$$I(p_j) = D_j,$$

and let I act identically on arithmetic and commute with logical connectives and quantifiers. Every D_j is closed. In particular, I does *not* replace a quoted old formula code by the code of its interpretation. Therefore

$$I(R_j) = (\exists \pi V(\pi, \overline{2j+1})) \rightarrow D(\bar{j}), \quad (10)$$

which is an instance of the uniform B -theorem (9).

Lemma 3. *There is a fixed integer $C \geq 1$ such that any S -proof π of A , of written length m , can be translated into a B -proof of $I(A)$ of written length at most $C(m+1)$.*

Proof. Let J be the finite set of indices of primitive predicates actually written in π . Whole-expression abbreviations cannot create a new predicate token. Consequently every predicate in an expanded record belongs to J , and

$$\sum_{j \in J} (1 + \text{bits}(j)) \leq c_0(m+1)$$

for a fixed encoding constant c_0 . This charges each distinct index to one of its literal occurrences in the input; it does not bound each index by m and then unnecessarily pay that bound m times.

For each $j \in J$, introduce a fresh B -abbreviation d_j for $D(\bar{j})$. Its name is a binary-indexed abbreviation name of length $O(1 + \text{bits}(j))$, not a new primitive symbol. The total cost of these

definitions is $O(m + 1)$. Translate the original definitions in order and the original records by replacing p_j with d_j while retaining references to earlier definitions. This costs $O(m + 1)$ characters: each replacement name has at most a constant multiple of the source predicate name's length.

Freshness can be implemented without a $\log m$ factor. Reserve disjoint fixed residue classes of binary abbreviation indices for renamed source names, the names d_j , and the finitely many names used by fixed proof templates. Sending an index i to $qi + r$, for fixed q, r , increases its self-delimiting binary name length by at most a constant factor. No original name is assumed to have constant size.

Before processing the translated input records, include one copy of the fixed proof of (9). For each $j \in J$, instantiate that theorem once at $\bar{j}$ and retain the resulting proof of

$$I(R_j) = (\exists \pi V(\pi, \overline{2j + 1})) \rightarrow D(\bar{j}).$$

The canonical numeral for $2j + 1$ is $b_1(\bar{j})$. Thus this preparation costs $O(1 + \text{bits}(j))$ characters per index, and $O(m + 1)$ altogether. The preparation uses only the new definitions, so it can precede all source definitions. It establishes each needed reflection image once, even if the source proof invokes that axiom repeatedly.

Logical and arithmetic axioms translate to axioms; induction instances remain induction instances. Modus ponens and generalization remain valid, and their side conditions are preserved because the replacements D_j are closed. Replace an extra-axiom record by a reiteration of its already proved image. No new rule is necessary: prove $F \rightarrow F$ by a fixed propositional template and apply modus ponens to the earlier F . Here F is written using the translated representation of that source record, including any abbreviations then in scope. The cost is a constant multiple of that record's written length. Premise lookup uses equality of expanded formulas, so the cached proof and this representation need not use the same abbreviation names. We use the permitted version of Hilbert records in which the verifier finds modus-ponens premises, rather than requiring explicit line references.

Summing these costs gives $C(m + 1)$. The last translated inference concludes $I(A)$, including when the source's last line is a reflection axiom. The argument bounds written proof length without expanding the definition graph. It does not assert a polynomial running time for the translator. $\square$

Theorem 4. *For the system S above there are fixed constants $a, b, C \geq 1$ such that, for every $n \geq 0$,*

$$|p_n| + 1 \leq b(1 + \text{bits}(n)), \quad (11)$$

$$\ell_S(\Box_S p_n \rightarrow p_n) \leq a(1 + \text{bits}(n)), \quad (12)$$

$$\ell_S(p_n) > n/C - 1, \quad (13)$$

where $\text{bits}(n)$ is the binary digit length, with $\text{bits}(0) = 1$. Moreover $\ell_S(\Box_S p_n \rightarrow p_n) \rightarrow \infty$ as $n \rightarrow \infty$. Thus a tail of this family satisfies (1).

Proof. The first bound follows from the binary coding of predicate names. The second is the length of the one-line axiom proof of R_n . The fixed arithmetic formula V contributes a constant; the numeral $\overline{2n + 1}$ and the name p_n each contribute $O(\log(n + 2))$. The finite check required to accept this line is true by Lemma 1, and its computation time is not proof-file length.

Let $m = \ell_S(p_n)$, which is finite by Proposition 2. Lemma 3 gives a B -proof of D_n of length at most $C(m + 1)$. Lemma 1 implies

$$n < \ell_B(D_n) \leq C(m + 1),$$

and hence (13).

For every fixed length bound M there are only finitely many proof files with at most M characters. Each has one designated conclusion. The sentences R_n are pairwise syntactically distinct, so only

finitely many can have such a proof. This proves that their least proof lengths tend to infinity, including in the presence of abbreviations.

Finally n eventually dominates every fixed multiple of $1 + \text{bits}(n)$. The lower bound therefore eventually exceeds $2a(1 + \text{bits}(n)) + C_1b(1 + \text{bits}(n))$, which implies (1). $\square$

An explicit rule selecting the witness sequence. The proof need not leave the choice of a tail informal. Fix integer constants a, b, C, C_1 from the displayed proof templates and translation. Set

$$T(n) = (2a + C_1b)(1 + \text{bits}(n)).$$

Let n_0 be the least positive integer satisfying $n_0 \geq C(T(n_0) + 1)$, and let n_{i+1} be the least integer greater than n_i satisfying the same decidable inequality. These searches terminate because $n/(1 + \text{bits}(n)) \rightarrow \infty$. Put $P_i = p_{n_i}$. If an S -proof of p_{n_i} had length at most $T(n_i)$, its interpretation would contradict $n_i < \ell_B(D_{n_i})$. Thus

$$\ell_S(P_i) > T(n_i) \geq 2\ell_S(\Box_S P_i \rightarrow P_i) + C_1(|P_i| + 1).$$

This specifies an effective family and the concrete factor $\delta = 1$; numerical values of the constants await an implementation of the fixed proof templates, rather than being asserted without a character ledger.

6 Adversarial checks and the limits of the result

No axiom is accepted by an oracle. The only unusual acceptance condition is the finite computation $H(j)$. It halts even if the consistency assumption is dropped. Consistency is used to show that its standard output is true, not to make it computable.

The proof predicate is the right one. Equation (7) is a PA theorem identifying V with the actual parameterized verifier at its fixed description. The reflection axioms use $\Box_S$, not provability in a stronger background theory or an unaligned surrogate. The derivability conditions were checked separately.

The use of Löb is legitimate. Löb is invoked after consistency, the actual accepted reflection proofs, and the derivability conditions have been established. It is not used to justify acceptance of an axiom or the finite Gödel lower bound.

The lower bound quantifies over every direct proof. The interpretation applies to an arbitrary S -proof, including one using other reflection axioms, induction, or an unexpected shortcut. Its output is a forbidden short proof of D_n if the input is too short. Merely comparing two particular proof scripts would not establish this.

Abbreviations have not been silently expanded. The translation preserves the definition graph. Closed replacements preserve binding conditions. Identifier characters are charged. Restricting abbreviations to complete expressions is a declared choice of system and prevents a short file from manufacturing a primitive predicate name of exponentially many digits.

The bottleneck is internalizing an accepted proof. A one-line proof of R_n is accepted after the external checker completes $H(n)$. To prove internally that this particular file is accepted, arithmetic must establish the relevant check. The existence of a primitive recursive checker gives no polynomial bound on the length of that internal verification. The example exploits precisely this missing requirement.

Although every standard test $H(n)$ succeeds, B cannot prove $\forall n H(n)$. Otherwise (2) would give a fixed proof of $\forall n D(n)$, whose numeral instances would have $O(\log(n + 2))$ -length proofs, contrary to Lemma 1. Consequently one cannot simply erase the guard inside arithmetic because it is extensionally redundant. The choice of the actual verifier and its arithmetization is essential.

This is not a lower bound on the complexity of the *set* of accepted files. An externally equivalent algorithm can omit all the true tests and check only the underlying Hilbert derivation. Its arithmetic acceptance formula is then different. An external equality of accepted files does not supply a PA proof identifying that formula with V . Thus a slow specified algorithm, a polynomial-time decidable proof relation, and an efficiently internalizable provability formula are three different assertions.

Why this is not a solution for the default PA presentation. The system has new primitive sentence symbols, extra reflection axioms, and a slow verification condition. The ability to abbreviate expressions, by itself, does not license replacing the specified PA_{bin} by this system when answering a question that fixes PA_{bin} . Theorem 4 answers only the explicitly allowed chosen-system branch of MAIS-O11(1).

What follows for the overhead function. For $k_n = a(1 + \text{bits}(n))$ and $r_n = b(1 + \text{bits}(n))$, the definition in MAIS-A1 gives

$$F_S(k_n, r_n) \geq \ell_S(p_n) > n/C - 1.$$

Thus this particular F_S is not bounded by any polynomial in its two arguments. This shows that the three-clause definition of efficiency alone cannot force polynomial Löb overhead. It does *not* decide the conjecture for $F_{\text{PA}_{\text{bin}}}$ in MAIS-O11(2)/MAIS-O10.

What is not being claimed about the companion problems. The elementary runtime bound above is not a bound on the shortest arithmetic proof verifying acceptance. No tight expansion bound, internal budget certificate with explicit constants, FairBot threshold, or bounded-GL theorem is obtained here. The resource-bounded companion statements have further uniform certificate hypotheses. This note establishes the unbounded chosen-system separation; it does not substitute that result for those additional obligations. The chosen verification procedure is slow; no lower bound against all extensionally equivalent checking algorithms is asserted. Replacing its arithmetization by the one associated with a different checker changes the reflection formula under study. The arithmetic-only variant avoids the language extension, but fails uniform instantiation. No answer for the standard, fixed-verifier problem is asserted.

7 An arithmetic-only variant and an exact obstruction

The new predicate symbols can be avoided if one asks only for the unbounded Löb conditions and the three-clause definition of efficiency. This variant has exactly the same accepted proof files as B , under the consistency assumption. It nevertheless uses a different checker and a different arithmetization of provability. It fails the agenda's additional uniform instantiation requirement. Both facts are essential to its interpretation.

7.1 A recognizable hard arithmetic family

Reserve the arithmetic variable z_0 . A variant of the first diagonal construction gives a fixed formula

$$G(x) = \forall z_0 (x = x \wedge E(x))$$

with z_0 absent from E , and a PA-provably Δ_1 predicate K , such that

$$B \vdash \forall x (G(x) \leftrightarrow K(x)), \quad K(n) \equiv \text{no } B\text{-proof of } G(\bar{n}) \text{ has length at most } n. \quad (14)$$

For clarity, this is not an arbitrary syntactic modification of an existing fixed point. Apply the parameterized diagonal lemma to the operator that, given a formula code e , first wraps that formula as $\forall z_0 (x = x \wedge \cdot)$ and then tests for short proofs of its numeral instances. The resulting E asserts the absence of a short proof of the wrapped formula G . Since the wrapper is provably equivalent to E , this gives (14). The diagonal construction can use variables other than z_0 . The visible conjunct $x = x$ ensures that different canonical numeral instances $G_n := G(\bar{n})$ are syntactically distinct.

Exactly as in Lemma 1, consistency gives, for every standard n ,

$$K(n), \quad B \vdash G_n, \quad \ell_B(G_n) > n. \quad (15)$$

In particular $B \not\vdash \forall n K(n)$: such a theorem, combined with (14), would give $O(\log(n+2))$ -length proofs of all the G_n by numeral instantiation, contradicting the last inequality.

Use a new formula coding c_* in which $c_*(G_n) = 2n+1$ and every other formula has an even ordinary string code. This coding is injective, primitive recursive, and has linear-size quoted numerals. The base coding used inside K is unchanged. This separation of the two codings avoids a circular definition of G .

7.2 The actual arithmetic checker

Define $\text{Prf}_*(\pi, y)$ to mean the following. Decode y using c_* ; check that π is a B -proof of that formula; expand its definitions; and, for every subformula of every expanded record or definition that is exactly a canonical instance G_n , run the finite test $K(n)$ and require success. This is a primitive recursive, PA-provably terminating algorithm. All these tests succeed on standard inputs by (15). Thus, if S_* is the system checked by this algorithm, then

$$\{S_*\text{-proof files}\} = \{B\text{-proof files}\}, \quad \ell_{S_*}(A) = \ell_B(A). \quad (16)$$

The equality is an external theorem from consistency. It is not an equivalence of the two proof predicates provable in B .

Choose a Σ_1 presentation $V_*(u, y)$ of this PA-provably Δ_1 checker, renaming all its variables to avoid z_0 , and set $\Box_* A := \exists u V_*(u, c_*(A))$. The existential binder also avoids z_0 . No quoted formula occurs literally inside its quotation, so *every* formula $\Box_* A$ contains no variable token z_0 .

The guard gives a fixed uniform theorem

$$B \vdash \forall n ((\exists u V_*(u, b_1(n))) \rightarrow G(n)). \quad (17)$$

Indeed an accepted proof with conclusion code $2n+1$ has conclusion G_n , so its conclusion itself triggers the test $K(n)$. Use (14) to finish. Instantiating the fixed proof at $\bar{n}$ costs $O(\log(n+2))$ characters. By (16), those are also S_* -proofs. Hence

$$\ell_{S_*}(\Box_* G_n \rightarrow G_n) = O(\log(n+2)), \quad \ell_{S_*}(G_n) > n. \quad (18)$$

Reflection lengths tend to infinity by the same finite-file argument as before. Appending a cheap reflection proof introduces no new guarded subformula: $\Box_* A$ has no z_0 , and all guarded subformulas in A were already checked in the given proof. Thus the additive toll also remains available, and a tail of this family satisfies the requested $\delta = 1$ inequality.

7.3 Why Löb’s theorem still applies

This is not merely a predicate chosen to make a reflection statement easy. The following verifies all three unbounded derivability conditions.

1. An actual accepted proof has a finite acceptance computation verifiable in B . By (16), the resulting theorem is also an S_* -theorem. This is external necessitation.
2. Concatenate accepted proofs of $A \rightarrow F$ and A , rename abbreviation identifiers to avoid collisions, and append F by modus ponens. The appended formula F is already a subformula of the first input’s conclusion. Thus no new G_n test is required. PA proves this syntactic fact and consequently proves the distribution axiom for $\Box_*$. The proof itself belongs to S_* by (16).
3. Formalized Σ_1 -completeness for the ordinary base checker gives

$$B \vdash \Box_* A \rightarrow \Box_B(\Box_* A).$$

It remains to embed these particular B -proofs into S_* , without assuming a general proof-predicate equivalence. Fix a finite set of variable names containing every variable in the template $\exists u V_*(u, y)$ and excluding z_0 . There is a primitive recursive injective renaming of all arithmetic variables which fixes this finite set and has z_0 outside its image. For example, fix indices $1, \dots, M$, send 0 to $M+1$, and send $i > M$ to $i+1$. Apply this renaming throughout a B -proof of $\Box_* A$ and its definitions. Logical axioms, PA induction, and inference rules remain valid; the conclusion is unchanged. The renamed file has no z_0 , so it has no subformula G_n and all additional tests are vacuous. PA proves correctness of this transformation. Therefore

$$B \vdash \Box_B(\Box_* A) \rightarrow \Box_*(\Box_* A),$$

and hence $S_* \vdash \Box_* A \rightarrow \Box_* \Box_* A$.

The diagonal lemma is available for the new effective coding. These facts establish applicability of ordinary Löb’s theorem. The proof of the separation itself already supplies theorems on both sides independently.

7.4 The exact uniform property that fails

Let $\tau(x) := G(x) \rightarrow G(x)$. There is a fixed S_* -proof of $\forall x \tau(x)$, since it is a B -theorem and the accepted proof files coincide. But

$$S_* \not\vdash \forall n \Box_* \tau(\bar{n}). \tag{19}$$

The expression here denotes the usual arithmetized numeral-instance coding. To prove the claim, an accepted proof of $\tau(\bar{n})$ contains G_n as a subformula of its conclusion. PA therefore proves uniformly that $\Box_* \tau(\bar{n}) \rightarrow K(n)$. A theorem excluded by (19) would give a B -proof of $\forall n K(n)$, contradicting (15).

Thus even the unbounded conclusion of the agenda’s quantifier-distribution principle fails here; its bounded version cannot hold. This variant answers only the literal unbounded, chosen-presentation question. It must not be used to instantiate the stronger resource-bounded hypotheses. The main construction avoids this particular obstruction: substituting a numeral for an arithmetic variable does not create a new nullary predicate symbol.

The possibility that externally identical theories have internally different provability predicates is familiar from the slow-provability literature; see Henk and Pakhomov [5]. That literature does not, by itself, establish the particular size separation above. The finite tests, syntactic renaming argument, and lower bound are the required additional ingredients.

8 Rejected mechanisms and a precise formalization target

Two simpler constructions do not answer the intended question. First, requiring extra padding only for direct conclusions creates a formal size gap but can destroy cheap proof composition. The present construction adds no such padding. Second, a finite Gödel sentence by itself supplies a lower bound on $\ell_B(D_n)$, but supplies no short proof of $\Box_B D_n \rightarrow D_n$. The new verifier and interpretation are what bridge that missing step in the chosen system.

Another attempted route is to make a sentence assert that every proof of it has a sufficiently shorter reflection proof. Consistency may then enforce the desired comparison *if the sentence is provable*, but it does not establish that provability. Unprovability of both sentences is compatible with the proposed comparison. The present construction supplies actual reflection proofs and checks Löb’s applicability instead.

For a Lean development, the following are separate mathematical obligations:

1. Define the base and extended syntax, proof files, expansion, actual character lengths, and the primitive recursive checkers.
2. Formalize the uniform finite diagonal family, prove the hard-sentence lemma from consistency, and formalize the second diagonal construction for V . These require syntax and arithmetization, not merely a modal box axiom.
3. Prove the guard invariant, consistency/conservativity, and the derivability conditions for the actual checker.
4. Implement the interpretation on compressed proof files and verify its length bound. Prove that it handles every admitted rule and side condition.
5. Derive the explicit family inequality and audit all assumptions of the final theorem. Consistency of the chosen PA presentation must be supplied or proved in the metatheory; it must not disappear behind an undeclared axiom.

Formalizing only the last inequality while assuming the guard and translation lemmas would certify a conditional transfer theorem, not the whole construction. This note contains no Lean files and no claim that Lean has checked it.

References

- [1] *Quantitative bounded Löb: proof-length overheads in Löb’s theorem and Löbian cooperation*, MAIS-A1, uploaded version; and *Does Löb’s rule shorten proofs?*, MAIS-O11, uploaded version. The chosen-system permission is in Question 3.6(1) / MAIS-O11(1).
- [2] A. Critch, *A parametric, resource-bounded generalization of Löb’s theorem, and a robust cooperation criterion for open-source game theory*, Journal of Symbolic Logic 84 (2019), 1368–1381. <https://arxiv.org/abs/1602.04184>.
- [3] P. Pudlák, *Incompleteness in the finite domain*, Bulletin of Symbolic Logic 23 (2017), 405–441. <https://arxiv.org/abs/1601.01487>.
- [4] P. Pudlák, *The lengths of proofs*, in S. R. Buss (ed.), Handbook of Proof Theory, Elsevier, 1998, 547–637. Section 2.1 uses polynomial-time decidability of the proof relation. That extensional complexity property must be distinguished from the choice of an arithmetic formula representing the relation. <https://users.math.cas.cz/~pudlak/length.pdf>.
- [5] P. Henk and F. Pakhomov, *Slow and Ordinary Provability for Peano Arithmetic*, 2016 preprint, version 2. <https://arxiv.org/abs/1602.01822>.